Concurrent Programming: Languages and techniques

Rust “fearless” concurrency

António Ravara

(with Bernardo Toninho and Carla Ferreira)

Departament of Computer Science,
NOVA School of Science and Technology

Concurrency on Rust – safely relaxing linearity*

Ownership discipline can be over-restrictive

Not possible to implement a doubly-linked list
(nodes are aliased by neighbours)

Instead of complicating or giving away, the ownership discipline is relaxed inside *safe* APIs

- Controlled use of aliased state – *unsafe code within safe APIs*
- Preserve memory safety across multiple threads

What do we mean by *safe*:

- memory-safe, as before
- race-free

* "Safe Systems Programming in Rust". Ralf Jung *et al.*"
DOI 10.1145/3418295

Rust is agnostic on how to do concurrency

No preference:

- supports shared memory and message-passing
  (in this course we deal with the former)

How can there be "races" (interference)?

We can put code running in parallel in *threads*

What's a data race?

- two or more *threads* (parallel pieces of running code)
  access the same memory location
- one or more of them writes in the location
- one or more of them is *unsynchronised* – may read or write at the same time others are reading or writing
  (there is no way of imposing an ordering between them)

Threads

- Rust standard library uses a 1:1 model of threads (one op. sys. thread per application thread)
- There are crates that implement other models of threading that make different tradeoffs to the 1:1 model
- When the main thread of a Rust program completes, all spawned threads are shut down

Thread Spawing

How to launch code to run in parallel?

With "*closures*" (more on them soon)

```rust
use std::thread;
use std::time::Duration;

fn main() {
    thread::spawn(|| {
        for i in 1..10 {
            println!("hi number {} from the spawned thread!", i);
            thread::sleep(Duration::from_millis(1));
        }
    });

    for i in 1..5 {
        println!("hi number {} from the main thread!", i);
        thread::sleep(Duration::from_millis(1));
    }
}
```

Thread Join

How to get results from code running in parallel?

`spawn` returns a `JoinHandle` – owned value with `join` "method"

When called, method `join` waits for its thread to finish

```
fn main() {
    let handle = thread::spawn(|| {
        for i in 1..10 {
            println!("hi number {} from the spawned thread!", i);
            thread::sleep(Duration::from_millis(1));
        }
    });

    for i in 1..5 {
        println!("hi number {} from the main thread!", i);
        thread::sleep(Duration::from_millis(1));
    }

    handle.join().unwrap();
}
```

Thread Move

```rust
fn main() {
    let v = vec![1, 2, 3];

    let handle = thread::spawn(|| {
        println!("Here's a vector: {:?}", v);
    });

    drop(v);

    handle.join().unwrap();
}
```

- `main` owns `v`
- `closure` borrows `v`

Error

compiler cannot tell how long the thread will run

Thread Move

```rust
fn main() {
    let v = vec![1, 2, 3];

    let handle = thread::spawn(move || {
        println!("Here's a vector: {:?}", v);
    });

    handle.join().unwrap();
}
```

- `move` closures *take ownership* of captured values
- `v` cannot be accessed in main anymore

On Closures in Rust

Closures capture their environment: variables, references, and their values

`unwrap_or_else` is a method of built-in sum types

`Option::unwrap_or_else()` does not expect any parameter for its closure (it can only be None)

`Result::unwrap_or_else()` passes `E` from `Err(E)` to the closure

```rust
impl Inventory {
    fn giveaway(&self, user_preference: Option<ShirtColor>) -> ShirtColor {
        user_preference.unwrap_or_else(|| self.most_stocked())
    }

    fn most_stocked(&self) -> ShirtColor {
        let mut num_red = 0;
        let mut num_blue = 0;

        for color in &self.shirts {
            match color {
                ShirtColor::Red => num_red += 1,
                ShirtColor::Blue => num_blue += 1,
            }
        }
        if num_red > num_blue {
            ShirtColor::Red
        } else {
            ShirtColor::Blue
        }
    }
```

```rust
enum ShirtColor {
    Red,
    Blue,
}

struct Inventory {
    shirts: Vec<ShirtColor>,
}
```

(Im)mutable Borrows in Closures

Closures borrow values by default

```rust
fn main() {
    let list = vec![1, 2, 3];
    println!("Before defining closure: {:?}", list);

    let only_borrows = || println!("From closure: {:?}", list);

    println!("Before calling closure: {:?}", list);
    only_borrows();
    println!("After calling closure: {:?}", list);
}
```

Closures capture mutable references

```rust
fn main() {
    let mut list = vec![1, 2, 3];
    println!("Before defining closure: {:?}", list);

    let mut borrows_mutably = || list.push(7);

    borrows_mutably();
    println!("After calling closure: {:?}", list);
}
```

Between the closure definition and its call, NO immutable borrows

Move ownership in Closures

Move transfers to closures ownership of the values it uses

```
use std::thread;

fn main() {
    let list = vec![1, 2, 3];
    println!("Before defining closure: {:?}", list);

    thread::spawn(move || println!("From thread: {:?}", list)
        .join()
        .unwrap();
}
```

Omitting `move` is a compiler error.

Traits a closure implements

Closures captures and handles values from the environment

- `FnOnce` – can be called only once
  closures that move captured values out of their body will only implement `FnOnce` and none of the other `Fn` traits
- `FnMut` – can be called many times
  closures that don't move captured values out of their body, but that might mutate the captured values
- `Fn` – can be called many times
  closures that don't move captured values out of their body and that don't mutate captured values, as well as closures that capture nothing from their environment

Traits a closure implements

`unwrap_or_else` **can only call** `f` **once**

```
impl<T> Option<T> {
    pub fn unwrap_or_else<F>(self, f: F) -> T
    where
        F: FnOnce() -> T
    {
        match self {
            Some(x) => x,
            None => f(),
        }
    }
}
```

Traits a closure implements

`sort_by_key` **takes an** `FnMut` **closure: called multiples times**

Once for each item

```
struct Rectangle {
    width: u32,
    height: u32,
}
fn main() {
    let mut list = [
        Rectangle { width: 10, height: 1 },
        Rectangle { width: 3, height: 5 },
        Rectangle { width: 7, height: 12 },
    ];

    list.sort_by_key(|r| r.width);
    println!("{:#?}", list);
}
```

Traits a closure implements

Count number of times `sort_by_key` is called

```
fn main() {
    let mut list = [
        Rectangle { width: 10, height: 1 },
        Rectangle { width: 3, height: 5 },
        Rectangle { width: 7, height: 12 },
    ];

    let mut num_sort_operations = 0;
    list.sort_by_key(|r| {
        num_sort_operations += 1;
        r.width
    });
    println!("{:#?}, sorted in {num_sort_operations} operatio
}
```

A digression – the critical section problem

Consider N threads executing in an infinite loop a sequence of instructions and sharing some variables:

loop

non-critical

pre-protocol

critical

post-protocol

The critical section

a portion of code accessing some *shared writable resources*

A digression – the critical section problem

Assumptions:

- No variable used in the critical and non-critical sections are used in the protocols, and vice-versa
- The critical section must progress;
  the non-critical needs not to.

The critical section problem:
At most one thread at a time may be executing its critical section

The goal:
Primitives to implement protocols to solve the problem

(to *synchronise* the threads)

A digression – the critical section problem

Envisaged properties:

- Mutual exclusion
  Statements in the critical section of two or more threads must *not* be interleaved
- Deadlock freedom
  If some threads are trying to enter their critical section, one must eventually succeed
- Starvation freedom
  If any thread is trying to enter its critical section, that thread must eventually succeed

Mutex: an API for *mut*ual *ex*clusion

Shared memory concurrency like multiple ownership

- multiple threads can access the same memory location at the same time
- management of shared memory (often) done via a `Mutex<T>` value

Mutexes guard data via locks

Management usually hard, but Rust's type system and ownership rules make it impossible to get it wrong

- blocks until it can acquire the exclusive lock
- implicitly gets released by the destructor when the variable goes out of scope

Mutex API safely controls shared references

```rust
use std::sync::Mutex;

fn main() {
    let m = Mutex::new(5);

    {
        let mut num = m.lock().unwrap();
        *num = 6;
    }

    println!("m = {:?}", m);
}
```

- After `lock` acquired, `num` is a mutable reference
- When `num` goes out of scope, the lock is released (via Drop)

Mutex: controls shared references – *memory and thread safe*

Thread safety: mutual exclusive access to shared memory

```
let mv = Mutex::new(vec![10, 11]);
join(
|| {let mut v= mv.lock().unwrap();v.push(12);},
|| {let mut v= mv.lock().unwrap();println!(``{:?}'',*v);}
);
```

prints either [10, 11, 12] if the first *thread* acquires first the *lock*,

or [10, 11] if it is the second who acquires it first

Both threads temporarily have exclusive access to the vector
`let mut v= mv.lock().unwrap()`

lock ensures mutual exclusion:
they will never both have a mutable reference *at the same time* –

uniqueness maintained

How to deal with shared references?

In the previous example, the scope of the reference is static

- When its "lifetime" ends, the reference is deleted and the original value is again uniquely owned (in subsequent code)
- This is ok with *structured parallelism* (like **join**)

What if the *threads* are independent from their "father"?

unstructured parallelism – threads are spawned off and keep running independently from the parent thread.

Note: `Mutex` **does not distinguish readers and writers;** `RwLock` **does**

Mutexes share values between multiple threads

Mutex offers mutation of aliased state safely

during each mutation, the state is not aliased

```
fn main() {
    let counter = Mutex::new(0);
    let mut handles = vec![];

    for _ in 0..10 {
        let handle = thread::spawn(move || {
            let mut num = counter.lock().unwrap();

            *num += 1;
        });
        handles.push(handle);
    }

    for handle in handles {
        handle.join().unwrap();
    }

    println!("Result: {}", *counter.lock().unwrap());
}
```

Error: move ownership of lock counter into multiple threads

Counting shared values between multiple threads

```rust
fn main() {
    let counter = Rc::new(Mutex::new(0));
    let mut handles = vec![];

    for _ in 0..10 {
        let counter = Rc::clone(&counter);
        let handle = thread::spawn(move || {
            let mut num = counter.lock().unwrap();

            *num += 1;
        });
        handles.push(handle);
    }

    for handle in handles {
        handle.join().unwrap();
    }

    println!("Result: {}", *counter.lock().unwrap());
}
```

Compiler error

We can't wrap the mutex in a reference counted (`Rc`) smart pointer: it cannot be sent between threads safely, as it has no interference control (changes can happen in several threads simultaneously)

Counting shared values between multiple threads

`Arc`: thread-safe version of `Rc`

Atomic reference counted smart pointer

```rust
use std::sync::{Arc, Mutex};
use std::thread;

fn main() {
    let counter = Arc::new(Mutex::new(0));
    let mut handles = vec![];

    for _ in 0..10 {
        let counter = Arc::clone(&counter);
        let handle = thread::spawn(move || {
            let mut num = counter.lock().unwrap();

            *num += 1;
        });
        handles.push(handle);
    }

    for handle in handles {
        handle.join().unwrap();
    }

    println!("Result: {}", *counter.lock().unwrap());
}
```

Types for "basic" atomic values

`std::sync::atomic` provides atomic versions of a select number of primitive types (*e.g.* `AtomicBool`, `AtomicI32`, etc.)

Atomically reference-counted pointers

For shared data

a mutable reference cannot be obtained, but a shared one can

The compiler ensures

alias got via `Arc` not destroyed during the lifetime of the ref

`Arc` **references:**

- `Arc<T>` points to `T`, and counts how many aliases exist
- when the last alias is deleted, the original ref is mutable again

```
let arc_v1 = Arc::new(vec![10, 11]);
let arc_v2 = Arc::clone(&arc_v1);//move transfers owner
spawn(move || println!(“{:?}”, arc_v2[1]));
    println!(“{:?}”, arc_v1[1]);
```